



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	SW-AyLP-APE-04: Clases Objetos y Métodos
Unidad de Organización Curricular:	ACADÉMICA
Nivel y Paralelo:	1 - B
Alumnos participantes:	Balarezo Pérez Diego Sebastián
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar programas utilizando clases, objetos y métodos, lo cual permite disminuir errores reutilizar el código.

Específicos:

- Desarrollar un código de sistema de notas para estudiantes en donde se ingresen datos tales como el nombre, la cédula, las notas y se calculen los promedios.
- Utilizar lo aprendido en POO al momento de crear el código aplicando clases, métodos y funciones.
- Explicar las partes del código mediante comentarios para una mejor comprensión del mismo.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- La tarea se debe realizar de forma individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.

- Codifique en Java cada uno de los ejercicios planteados.
- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios. Las capturas se subirán en un archivo pdf en el formato de Guías APE.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒ Plataformas educativas

☐ Simuladores y laboratorios virtuales

☒ Aplicaciones educativas

☐ Recursos audiovisuales

☐ Gamificación

☒ Inteligencia Artificial

Otros (Especifique): -Computadora o laptop, -Internet para consultas, -Papel, -Material de escritorio.

2.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas

2.7 Resultados obtenidos

Los resultados obtenidos durante el desarrollo de los ejercicios fueron satisfactorios, ya que se logró aplicar correctamente la búsqueda lineal para localizar productos dentro de arreglos paralelos y mostrar sus respectivos precios de manera eficiente. Además, se verificó que el control adecuado de los límites en los ciclos permitió evitar errores relacionados con el acceso indebido a la memoria, garantizando así la estabilidad y seguridad del programa. También se comprendió la importancia de prevenir el desbordamiento de búfer mediante validaciones y comentarios técnicos que ayudan a mantener un código más seguro y organizado

2.8 Habilidades blandas empleadas en la práctica

☐ Liderazgo

☐ Trabajo en equipo

☐ Comunicación asertiva

☐ La empatía

- ✗ Pensamiento crítico
- ✗ Flexibilidad
- ☐ La resolución de conflictos
- ✗ Adaptabilidad
- ✗ Responsabilidad

2.9 Conclusiones

El uso de la programación orientada a objetos permite disminuir errores en el código y la reutilización de código.

2.10 Recomendaciones

- Previo a la elaboración de las tareas el estudiante debe revisar el material proporcionado y la bibliografía.
- Si se manifiestan dudas sobre el tema de la APE, se invita a los estudiantes acudir a las tutorías académicas.
- Comentar el código.

2.11 Anexos

2.11.1 Ejercicio 1

2.11.1.1 Enunciado

Sistema básico de control de estudiantes y calificaciones

Desarrollar un programa en C++ y Java que permita registrar información de estudiantes de la asignatura Algoritmos y Lógica de Programación. El sistema debe aplicar programación orientada a objetos mediante clases, objetos y métodos.

- Cada estudiante debe tener:
 - Cédula
 - Nombre
 - Apellido
 - Nota 1
 - Nota 2
 - Nota 3
 - Promedio

Estado: Aprobado o Reprobado

2.11.1.2 Análisis del problema

El propósito de este programa es crear un sistema de ingreso de notas en el cuál se pueda ingresar datos de los estudiantes tales como su nombre y apellido, sus notas y su cédula de identidad por cada estudiante. También el ejercicio propuesto es una excelente práctica para poder mejorar las habilidades y el uso de POO con clases, objetos y métodos utilizados dentro del código; también se debe tomar en cuenta ciertas condiciones establecidas al momento de la codificación.

Condiciones:

1. Registrar mínimo 5 estudiantes
2. Ingresar las 3 notas de cada estudiante
3. Calcular automáticamente el promedio
4. Mostrar el listado completo de estudiantes
5. Mostrar cuántos estudiantes aprobaron
6. Mostrar cuántos estudiantes reprobaron
7. Validar que las notas estén entre 0 y 10

Comentar correctamente el código

2.11.1.3 Declaración de variables

Nombre de Variable	Tipo de Dato (C++)	Tipo de Dato (Java)	Descripción
cedula	string	String	Variable para almacenar las cédulas de los estudiantes en arreglos posteriores.
nombre	string	String	Variable para almacenar los nombres de los estudiantes en arreglos posteriores.

apellido	string	String	Variable para almacenar los apellidos de los estudiantes en arreglos posteriores.
nota1	double	double	Variable para almacenar la nota 1 de los estudiantes en arreglos posteriores.
nota2	double	double	Variable para almacenar la nota 2 de los estudiantes en arreglos posteriores.
nota3	double	double	Variable para almacenar la nota 3 de los estudiantes en arreglos posteriores.
promedio	double	double	Variable para almacenar el promedio de los estudiantes en arreglos posteriores.
estado	string	String	Variable para almacenar el estado académico de los estudiantes en arreglos posteriores.

2.11.1.4 Algoritmo paso a paso

Inicio

Definir la clase Estudiante.

Definir las variables cédula, nombre, apellido, nota1, nota2, nota3, promedio, estado.

Definir el constructor de forma pública con las variables establecidas.

Definir getters y setters para obtener y establecer las variables correspondientes con valores.

Definir una función para calcular el promedio.

Definir una función para determinar el estado de aprobado o reprobado.

Definir una función para poder imprimir la información de manera más eficiente.

Definir una función para verificar las notas que estén dentro del rango permitido.

Establecer la función principal o “main”.

Definir un vector estudiantes para un mejor manejo de información de las variables en general.

Establecer la cantidad de estudiantes.

Crear un ciclo “for” para solicitar y almacenar los datos.

Solicitar los datos al usuario.

Solicitar las notas (dentro del rango).

Llamar al objeto estudiante.

Guardar toda la información almacenada en un vector (para una mejor impresión y manejo de los datos).

Definir contadores “aprobados” y “reprobados”.

Crear ciclo “for” para definir la cantidad de estudiantes aprobados y reprobados.

Imprimir en consola el reporte necesario de la información.

Fin

2.11.1.5 Pseudocódigo

Algoritmo Registro_Estudiantes_Algoritmos

// Definición de variables

Definir cantidadEstudiantes, i, aprobados, reprobados Como Entero

Definir cedula, nombre, apellido, estado Como Cadena

Definir nota1, nota2, nota3, promedio Como Real

// Arreglos para almacenar datos

Dimension cedula[5]

Dimension nombres[5]

Dimension apellidos[5]

Dimension notas1[5]

Dimension notas2[5]

Dimension notas3[5]

Dimension promedios[5]

Dimension estados[5]

cantidadEstudiantes <- 5

aprobados <- 0

reprobados <- 0

Escribir "=== REGISTRO DE ESTUDIANTES - ALGORITMOS ==="

// Registro de estudiantes

Para i <- 0 Hasta cantidadEstudiantes - 1 Hacer

Escribir ""

Escribir "--- Datos del Estudiante ", i + 1, " ---"

Escribir "Ingrese Cedula:"

Leer cedula

Escribir "Ingrese Nombre:"

Leer nombre

Escribir "Ingrese Apellido:"

Leer apellido

// Validación Nota 1

Repetir

Escribir "Ingrese la Nota 1 (0 - 10):"

Leer nota1

Si $\text{nota1} < 0$ O $\text{nota1} > 10$ Entonces

Escribir "¡Nota invalida! Debe estar entre 0 y 10."

FinSi

Hasta Que $\text{nota1} \geq 0$ Y $\text{nota1} \leq 10$

// Validación Nota 2

Repetir

Escribir "Ingrese la Nota 2 (0 - 10):"

Leer nota2

Si $\text{nota2} < 0$ O $\text{nota2} > 10$ Entonces

Escribir "¡Nota invalida! Debe estar entre 0 y 10."

FinSi

Hasta Que $\text{nota2} \geq 0$ Y $\text{nota2} \leq 10$

// Validación Nota 3

Repetir

Escribir "Ingrese la Nota 3 (0 - 10):"

Leer nota3

Si $\text{nota3} < 0$ O $\text{nota3} > 10$ Entonces

Escribir "¡Nota invalida! Debe estar entre 0 y 10."

FinSi

Hasta Que $\text{nota3} \geq 0$ Y $\text{nota3} \leq 10$

// Calcular promedio

$\text{promedio} \leftarrow (\text{nota1} + \text{nota2} + \text{nota3}) / 3$


```

// Determinar estado
Si promedio >= 7 Entonces
    estado <- "Aprobado"
    aprobados <- aprobados + 1
SiNo
    estado <- "Reprobado"
    reprobados <- reprobados + 1
FinSi

// Guardar datos en arreglos
cedulas[i] <- cedula
nombres[i] <- nombre
apellidos[i] <- apellido
notas1[i] <- nota1
notas2[i] <- nota2
notas3[i] <- nota3
promedios[i] <- promedio
estados[i] <- estado

FinPara

// Mostrar resultados
Escribir ""
Escribir "=====
Escribir "      LISTADO DE ESTUDIANTES      "
Escribir "=====

Para i <- 0 Hasta cantidadEstudiantes - 1 Hacer

    Escribir Sin Saltar "Cedula: ", cedulas[i]
        Escribir Sin Saltar " | Estudiante: ", apellidos[i], " ", nombres[i]
        Escribir Sin Saltar " | Notas: [", notas1[i], ", ", notas2[i], ", ", notas3[i], "]"
        Escribir Sin Saltar " | Promedio: ", promedios[i]
        Escribir Sin Saltar " | Estado: ", estados[i]

FinPara

```

Escribir ""

Escribir "=====

Escribir "RESUMEN DE RENDIMIENTO"

Escribir "Total Estudiantes Aprobados: ", aprobados

Escribir "Total Estudiantes Reprobados: ", reprobados

Escribir "=====

FinAlgoritmo

2.11.1.6 Diagrama de flujo



2.11.1.7 Codificación C++

```
#include <iostream>
#include <vector>
#include <string>
#include <iomanip> // Para dar formato a los decimales (.precision)

using namespace std;

class Estudiante {
private:
    // Atributos privados
    string cedula;
    string nombre;
    string apellido;
    double nota1, nota2, nota3;
    double promedio;
    string estado;

public:
    // Constructor
    Estudiante(string _cedula, string _nombre, string _apellido, double _n1, double _n2, double _n3) {
        cedula = _cedula;
        nombre = _nombre;
        apellido = _apellido;
        nota1 = _n1;
        nota2 = _n2;
        nota3 = _n3;
        calcularPromedio(); // Se calcula automáticamente al instanciar
        determinarEstado(); // Se determina el estado automáticamente
    }

    // Métodos Get y Set
    string getCedula() { return cedula; }
    void setCedula(string c) { cedula = c; }

    string getNombre() { return nombre; }
```

```
void setNombre(string n) { nombre = n; }
```

```
string getApellido() { return apellido; }
```

```
void setApellido(string a) { apellido = a; }
```

```
double getNota1() { return nota1; }
```

```
void setNota1(double n1) { nota1 = n1; calcularTodo(); }
```

```
double getNota2() { return nota2; }
```

```
void setNota2(double n2) { nota2 = n2; calcularTodo(); }
```

```
double getNota3() { return nota3; }
```

```
void setNota3(double n3) { nota3 = n3; calcularTodo(); }
```

```
double getPromedio() { return promedio; }
```

```
string getEstado() { return estado; }
```

```
// Métodos de cálculo internos
```

```
void calcularPromedio() {
```

```
    promedio = (nota1 + nota2 + nota3) / 3.0;
```

```
}
```

```
void determinarEstado() {
```

```
    if (promedio >= 7.0) {
```

```
        estado = "Aprobado";
```

```
    } else {
```

```
        estado = "Reprobado";
```

```
    }
```

```
}
```

```
// Función auxiliar si se modifican notas mediante setters
```

```
void calcularTodo() {
```

```
    calcularPromedio();
```

```
    determinarEstado();
```

```
}
```

```

// Método para mostrar la información del estudiante
void mostrarInformacion() {
    cout << "Cedula: " << cedula
        << " | Estudiante: " << apellido << " " << nombre
        << " | Notas: [" << nota1 << ", " << nota2 << ", " << nota3 << "]"
        << " | Promedio: " << fixed << setprecision(2) << promedio
        << " | Estado: " << estado << endl;
}
};

// Función auxiliar para validar que la nota esté entre 0 y 10
double pedirNotaValidada(string numeroNota) {
    double nota;
    while (true) {
        cout << "Ingrese la " << numeroNota << " (0 - 10): ";
        cin >> nota;
        if (nota >= 0.0 && nota <= 10.0) {
            return nota;
        }
        cout << "¡Nota invalida! Debe estar entre 0 y 10. Intente de nuevo.\n";
    }
}

int main() {
    vector<Estudiante> listaEstudiantes;
    int cantidadEstudiantes = 5; // Requerimiento mínimo

    cout << "=== REGISTRO DE ESTUDIANTES - ALGORITMOS ===" << endl;

    for (int i = 0; i < cantidadEstudiantes; i++) {
        string cedula, nombre, apellido;
        double n1, n2, n3;

        cout << "\n--- Datos del Estudiante " << (i + 1) << " ---" << endl;
        cout << "Ingrese Cedula: ";
        cin >> cedula;
    }
}

```

```

cout << "Ingrese Nombre: ";
cin >> nombre;
cout << "Ingrese Apellido: ";
cin >> apellido;

// Validar cada nota mediante la función auxiliar
n1 = pedirNotaValidada("Nota 1");
n2 = pedirNotaValidada("Nota 2");
n3 = pedirNotaValidada("Nota 3");

// Crear el objeto y guardarlo en el vector
Estudiante nuevoEstudiante(cedula, nombre, apellido, n1, n2, n3);
listaEstudiantes.push_back(nuevoEstudiante);
}

// Reportes finales
cout << "\n===== " << endl;
cout << "      LISTADO DE ESTUDIANTES      " << endl;
cout << "===== " << endl;

int aprobados = 0;
int reprobados = 0;

for (int i = 0; i < listaEstudiantes.size(); i++) {
    listaEstudiantes[i].mostrarInformacion();

    if (listaEstudiantes[i].getEstado() == "Aprobado") {
        aprobados++;
    } else {
        reprobados++;
    }
}

cout << "\n===== " << endl;
cout << "RESUMEN DE RENDIMIENTO" << endl;
cout << "Total Estudiantes Aprobados: " << aprobados << endl;

```

```

cout << "Total Estudiantes Reprobados: " << reprobados << endl;
cout << "===== " << endl;

return 0;
}

```

2.11.1.8 Prueba de escritorio C++

```

=== REGISTRO DE ESTUDIANTES - ALGORITMOS ===

--- Datos del Estudiante 1 ---
Ingrese Cedula: 1850342724
Ingrese Nombre: Diego
Ingrese Apellido: Balarezo
Ingrese la Nota 1 (0 - 10): 7
Ingrese la Nota 2 (0 - 10): 9
Ingrese la Nota 3 (0 - 10): 9.5

--- Datos del Estudiante 2 ---
Ingrese Cedula: 1805695826
Ingrese Nombre: Mateo
Ingrese Apellido: Vargas
Ingrese la Nota 1 (0 - 10): 8
Ingrese la Nota 2 (0 - 10): 7
Ingrese la Nota 3 (0 - 10): 10

--- Datos del Estudiante 3 ---
Ingrese Cedula: 1800083416
Ingrese Nombre: Patricio
Ingrese Apellido: Ramirez
Ingrese la Nota 1 (0 - 10): 9
Ingrese la Nota 2 (0 - 10): 9
Ingrese la Nota 3 (0 - 10): 6

--- Datos del Estudiante 4 ---
Ingrese Cedula: 1856478934
Ingrese Nombre: Ramiro
Ingrese Apellido: Jacome
Ingrese la Nota 1 (0 - 10): 7
Ingrese la Nota 2 (0 - 10): 6
Ingrese la Nota 3 (0 - 10): 8

```

```

--- Datos del Estudiante 5 ---
Ingrese Cedula: 1857362659
Ingrese Nombre: Sebastian
Ingrese Apellido: Lopez
Ingrese la Nota 1 (0 - 10): 8
Ingrese la Nota 2 (0 - 10): 9
Ingrese la Nota 3 (0 - 10): 9

=====
LISTADO DE ESTUDIANTES
=====
Cedula: 1850342724 | Estudiante: Balarezo Diego | Notas: [7, 9, 9.5] | Promedio: 8.50 | Estado: Aprobado
Cedula: 1805695826 | Estudiante: Vargas Mateo | Notas: [8.00, 7.00, 10.00] | Promedio: 8.33 | Estado: Aprobado
Cedula: 1800083416 | Estudiante: Ramirez Patricio | Notas: [9.00, 9.00, 6.00] | Promedio: 8.00 | Estado: Aprobado
Cedula: 1856478934 | Estudiante: Jacome Ramiro | Notas: [7.00, 6.00, 8.00] | Promedio: 7.00 | Estado: Aprobado
Cedula: 1857362659 | Estudiante: Lopez Sebastian | Notas: [8.00, 9.00, 9.00] | Promedio: 8.67 | Estado: Aprobado

=====
RESUMEN DE RENDIMIENTO
Total Estudiantes Aprobados: 5
Total Estudiantes Reprobados: 0
=====
PS C:\Users\disba\Documents\Diegui\UTA Primer Semestre\VSCodeProjects>

```

2.11.1.9 Codificación Java

```

import java.util.Scanner;

// -----
// CLASE ESTUDIANTE
// -----

public class SistemasEstudiante {

    // Atributos privados: solo accesibles dentro de la clase
    private String cedula;
    private String nombre;
    private String apellido;
    private double nota1, nota2, nota3;
    private double promedio;
    private String estado;

    // — Constructor: inicializa el objeto con los datos recibidos —
    public SistemasEstudiante(String cedula, String nombre, String apellido,
                               double nota1, double nota2, double nota3) {
        this.cedula = cedula;
        this.nombre = nombre;
        this.apellido = apellido;
    }
}

```



```

this.nota1    = nota1;
this.nota2    = nota2;
this.nota3    = nota3;
calcularPromedio(); // Se calcula automáticamente al crear el objeto
determinarEstado();
}

// — Getters: lectura de atributos privados —
public String getCedula() { return cedula; }
public String getNombre() { return nombre; }
public String getApellido() { return apellido; }
public double getPromedio() { return promedio; }
public String getEstado() { return estado; }

// — Setters: modificación de atributos privados —
public void setNota1(double n) { this.nota1 = n; }
public void setNota2(double n) { this.nota2 = n; }
public void setNota3(double n) { this.nota3 = n; }

// — Calcula el promedio de las 3 notas —
public void calcularPromedio() {
    promedio = (nota1 + nota2 + nota3) / 3.0;
}

// — Determina si el estudiante aprueba (promedio >= 7.00) —
public void determinarEstado() {
    estado = (promedio >= 7.00) ? "Aprobado" : "Reprobado";
}

// — Muestra la información completa del estudiante —
public void mostrarInfo() {
    System.out.println("┌───────────────────────────────────────────────────────────────────────────────────┐");
    System.out.printf(" Cédula   : %s%n", cedula);
    System.out.printf(" Nombre   : %s %s%n", nombre, apellido);
    System.out.printf(" Notas    : %.2f | %.2f | %.2f%n", nota1, nota2, nota3);
    System.out.printf(" Promedio : %.2f%n", promedio);
}

```

```

System.out.printf(" Estado : %s%n", estado);
System.out.println(" _____");
}

// _____

// MÉTODO MAIN: punto de entrada del programa
// _____

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    final int TOTAL = 5;           // Mínimo 5 estudiantes

    SistemasEstudiante[] lista = new SistemasEstudiante[TOTAL]; // Arreglo de objetos

    System.out.println("=== REGISTRO DE ESTUDIANTES ===\n");

    // — Registro de cada estudiante —
    for (int i = 0; i < TOTAL; i++) {
        System.out.println("--- Estudiante " + (i + 1) + " ---");

        System.out.print(" Cédula : "); String ced = sc.next();
        System.out.print(" Nombre : "); String nom = sc.next();
        System.out.print(" Apellido : "); String ape = sc.next();

        double n1 = pedirNota(sc, "Nota 1");
        double n2 = pedirNota(sc, "Nota 2");
        double n3 = pedirNota(sc, "Nota 3");
        System.out.println();

        // Se crea el objeto y se guarda en el arreglo
        lista[i] = new SistemasEstudiante(ced, nom, ape, n1, n2, n3);
    }

    // — Listado completo —

    System.out.println("\n=== LISTADO DE ESTUDIANTES ===");
    int aprobados = 0, reprobados = 0;

    for (SistemasEstudiante e : lista) {

```

```

        e.mostrarInfo();
        // Conteo de aprobados y reprobados
        if (e.getEstado().equals("Aprobado")) aprobados++;
        else                                reprobados++;
    }

    // — Resumen final —
    System.out.println("\n=== RESUMEN ===");
    System.out.println(" Aprobados : " + aprobados);
    System.out.println(" Reprobados : " + reprobados);

    sc.close();
}

// —————
// Pide una nota válida (0–10), repite si es inválida
// —————
private static double pedirNota(Scanner sc, String label) {
    double nota;
    do {
        System.out.print(" " + label + " (0-10): ");
        nota = sc.nextDouble();
        if (nota < 0 || nota > 10)
            System.out.println(" ⚠ Nota inválida. Ingrese un valor entre 0 y 10.");
    } while (nota < 0 || nota > 10); // Repite hasta obtener valor válido
    return nota;
}
}

```

2.11.1.10 Prueba de escritorio Java

```

Nombre: Paolo
Apellido: Cordova
Ingrese la nota 1 (0 - 10): 8
Ingrese la nota 2 (0 - 10): 4
Ingrese la nota 3 (0 - 10): 5
-----
Estudiante #5:
Cedula: 48451215485
Nombre: Jaime
Apellido: Tapia
Ingrese la nota 1 (0 - 10): 8
Ingrese la nota 2 (0 - 10): 9
Ingrese la nota 3 (0 - 10): 10
-----

=== LISTADO COMPLETO DE ESTUDIANTES ===
Cedula: 1850337617 | Alumno: Cajiao Paulo | Notas: [8, 9, 7] | Promedio: 8 | Estado: Aprobado
Cedula: 187025487 | Alumno: Perez Juan | Notas: [8, 7, 6] | Promedio: 7 | Estado: Aprobado
Cedula: 0501805709 | Alumno: Guano Tania | Notas: [7, 6, 9] | Promedio: 7.33333 | Estado: Aprobado
Cedula: 175879652 | Alumno: Cordova Paolo | Notas: [8, 4, 5] | Promedio: 5.66667 | Estado: Reprobado
Cedula: 48451215485 | Alumno: Tapia Jaime | Notas: [8, 9, 10] | Promedio: 9 | Estado: Aprobado
=====

=== ESTADISTICAS DEL GRUPO ===
Estudiantes Aprobados (Promedio >= 7.00): 4
Estudiantes Reprobados (Promedio < 7.00): 1
=====

```

```

Nombre: Raul Menendez
Apellido: Ingrese la nota 1 (0 - 10): 9
Ingrese la nota 2 (0 - 10): 7
Ingrese la nota 3 (0 - 10): 10
-----
Estudiante #5:
Cedula: 9841653553
Nombre: Fernando Perez
Apellido: Ingrese la nota 1 (0 - 10): 9
Ingrese la nota 2 (0 - 10): 7
Ingrese la nota 3 (0 - 10): 2
-----

=== LISTADO COMPLETO DE ESTUDIANTES ===
Cedula: 1850337617 | Alumno: Cajiao Paulo | Notas: [8.0, 7.0, 9.0] | Promedio: 8.00 | Estado: Aprobado
Cedula: 776513133515 | Alumno: Tapia Jaime | Notas: [8.0, 9.0, 10.0] | Promedio: 9.00 | Estado: Aprobado
Cedula: Tania | Alumno: 6516516156 Guano | Notas: [4.0, 8.0, 3.0] | Promedio: 5.00 | Estado: Reprobado
Cedula: 784546565 | Alumno: Menendez Raul | Notas: [9.0, 7.0, 10.0] | Promedio: 8.67 | Estado: Aprobado
Cedula: 9841653553 | Alumno: Perez Fernando | Notas: [9.0, 7.0, 2.0] | Promedio: 6.00 | Estado: Reprobado
=====

=== ESTADISTICAS DEL GRUPO ===
Estudiantes Aprobados (Promedio >= 7.00): 3
Estudiantes Reprobados (Promedio < 7.00): 2
=====

```

2.11.2.9 Link de Github: <https://github.com/DiegoSW-08/APE04-Clases-Objetos-Metodos>